

Daniel Fraunholz, Daniel Reti, Marvin Sinnwell

Introduction

“Smashing the stack for fun and profit” is a famous phrack article from 1996 about the basics of stack overflow attacks from a time before mitigations like Data Execution Prevention [DEP], Address Space Layout Randomization [ASLR], Stack Canaries or Pointer Authentication Codes [PAC] existed. This exercise is meant to introduce you to the basics of stack overflow or buffer overflow attacks. <http://phrack.org/issues/49/14.html>

Exercise 1: Buffer Overrrun

Overview

After conducting reverse engineering tests during an ongoing penetration test, you encountered an authentication mechanism hosted on a remote server within the same engagement. Initially, you considered brute-forcing the login credentials as your first approach. However, you were informed that any form of brute-force attempt on the client’s assets was beyond the scope of the engagement.

Faced with this limitation, you experienced a moment of despair. However, your colleague examined the source code and provided you with a hint regarding a potential buffer overrun vulnerability that could be exploited to bypass authentication. She also pointed out that the developer used an unsafe function to accept input without proper boundary checks.

Unfortunately, due to her busy schedule with other tasks, she couldn’t offer further assistance. Now, it is your responsibility to analyze the security flaws and develop a proof-of-concept to bypass authentication.

Remote Server

The remote binary is hosted on your groups subnet on port 1337:

```
1 IP: 10.10.xx.246
2 Port: 1337
```

To establish a remote connection to the server binary using netcat, you can use the following command:

```
1 nc <ip_here> <port_here>
```

Task description

You have been given both the source code and the compiled binary of the authentication program. Your objective is to circumvent the authentication mechanism and gain unauthorized access to the secret information (referred to as the *FLAG* here) hidden within the program. The challenge is divided into two parts:

1. Local Exploitation

- (a) Examine the provided source code to comprehend the implementation of the authentication mechanism.

- (b) Analyze the code and identify any vulnerabilities or weaknesses that can be exploited.
- (c) Utilize the compiled binary on your local system to craft an input that allows you to bypass authentication and display the secret information (*FLAG*).

2. remote Exploitation

- (a) Establish a connection to the remote server using a network tool such as nc or Python's CTF framework and exploit development library *pwntools*.
- (b) The remote server will be running the same binary as the one provided to you. Your objective is to exploit it remotely to bypass the authentication.
- (c) Construct the appropriate input to achieve an authentication bypass and retrieve the secret information (*FLAG*).

Remember, you must bypass the authentication mechanism within the given constraints of the program. Do not modify the source code or the binary.

Hints and Directions

- Local Exploitation: For creating a local exploit, you have a couple of options. You can use the Linux pipe (|) in the command-line shell to connect the output of one command to the input of another command. This allows you to pass input to the binary. Alternatively, you can utilize pwntools, a powerful exploit development library, to interact with the binary locally and send input. You can find resources and documentation on pwntools to understand its usage and capabilities.
- Remote Exploitation: The method used for local exploitation can also be applied for remote exploitation. Instead of interacting with the local binary, you will need to send input to the remote server. You can employ the same pipe (|) technique, redirecting input to a nc (netcat) connection. Alternatively, you can leverage the remote feature of pwntools to communicate with the remote binary. The following pwntools-cheatsheet offers guidance on basic operations within pwntools. Furthermore, you can seek additional help and resources through online references for further assistance.
- Printing/Redirecting Payload: Both Bash and Python can be used to print or redirect the payload to the local or remote binary. If you opt for pwntools, you can directly use it to send your payload to the binary.

Exercise 2: Stack Smashing

Important: Both machines 10.10.XX.36 and 10.10.XX.82 take longer to reboot and reset. They can sometimes lose their DHCP lease. Reset and restart your network if you can't reach them. They take up to 5 minutes to reboot. Please excuse the inconvenience.

Overview

On your student lab you will find two machines. They are identical, except for the credentials. This means you can use the first machine to create a buffer overflow payload to exploit the second machine. The (SSH) credentials for the development machine are:

```
challenge03@challenge03:~/Desktop$ ls -la
total 16
drwxr-xr-x  2 challenge03 challenge03 4096 Feb 22 07:28 .
drwxr-xr-x 16 challenge03 challenge03 4096 May 30 05:11 ..
-rwxr-xr-x  1 root        root        7992 Feb 22 08:04 server
challenge03@challenge03:~/Desktop$ ./server
Socket successfully created..
Socket successfully binded..
Server listening..
```

Figure 1: Execution of the *server* binary from the remote lab machine.

```
root@kali:~# nc 192.168.56.110 10100
test
test
root@kali:~#
```

Figure 2: Remote connection to *server* binary using *netcat* on local Kali machine.

```
1 IP: 10.10.XX.36
2 Username: challenge04
3 Password: challenge04
```

You will find an executable binary called *server* on your desktop. Launching it will give you the message “Server listening”. You can remotely connect to the server from your kali, by connecting to the port 10100 (can be retrieved with *netstat* on the server machine or with port scanning), and sending a test message reveals that the server is simply echoing the message.

Setup

On the first machine, a graphical debugger called *edb* is installed. To attach the debugger, you can connect to the first machine with an SSH connection in graphical mode and type *edb*.

```
1 ssh -X challenge04@10.10.XX.36
2 Password: challenge04
```

Now execute the *server* binary which is placed on the desktop. And then start *edb* in another tab. A window should pop up with the debugger. A detailed walk through starting the debugger and triggering the buffer overflow is given in the Appendix section.

Of course you are free to use any other debugger you like, such as *gdb*.

The second machine is where you will gain root access after a successful buffer overflow.

```
1 IP: 10.10.XX.82
```

Task description

Before you continue, make sure you have understood the setup and how the buffer overflow is being caused and how the registers are affected.

The EIP (instruction pointer) tells the processor the address of the command to be executed

next. If we can control the EIP, we can control the execution flow. To receive a shell we need to generate shellcode (e.g. with MSFvenom), write the shellcode into the buffer, write the address of a 'jmp esp' command into the EIP (instruction pointer), which points to the buffer with the shellcode so it executed.

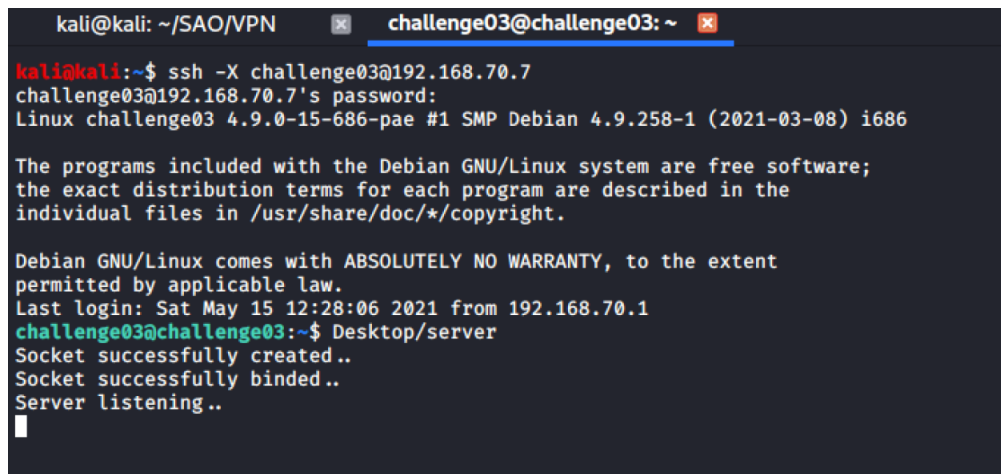
You can use our python template provided on OLAT to send the data. Here are the steps necessary to complete this task:

1. Find the correct offsets of the registers
2. Test for forbidden characters
3. Find an address with "jmp esp"
4. Create a shellcode with MSF venom
5. Write the jmp esp address into the EIP and the shellcode into the buffer
6. Check if the exploit is working and a shell is received
7. Use the exploit on the second machine 'ex4-2b'
8. Retrieve the flag in /root/flag.txt

Report

Write a brief report including your solutions and your findings. Then, submit the report via Olat. You can find a $\text{\LaTeX}$ template on OLAT.

Appendix: Setup the Debugger



```
kali@kali: ~/SAO/VPN challenge03@challenge03: ~
kali@kali:~$ ssh -X challenge03@192.168.70.7
challenge03@192.168.70.7's password:
Linux challenge03 4.9.0-15-686-pae #1 SMP Debian 4.9.258-1 (2021-03-08) i686

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat May 15 12:28:06 2021 from 192.168.70.1
challenge03@challenge03:~$ Desktop/server
Socket successfully created..
Socket successfully binded..
Server listening..
█
```

Figure 3: Start the *server* binary on the remote machine through an SSH connection.

```
kali@kali: ~/SAO/VPN challenge03@challenge03: ~ challenge03@challenge03: ~
kali@kali:~$ ssh -X challenge03@192.168.70.7
challenge03@192.168.70.7's password:
Linux challenge03 4.9.0-15-686-pae #1 SMP Debian 4.9.258-1 (2021-03-08) i686

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat May 15 12:28:53 2021 from 192.168.70.1
challenge03@challenge03:~$ edb
```

Figure 4: Start the graphical debugger *edb* from a separate SSH connection

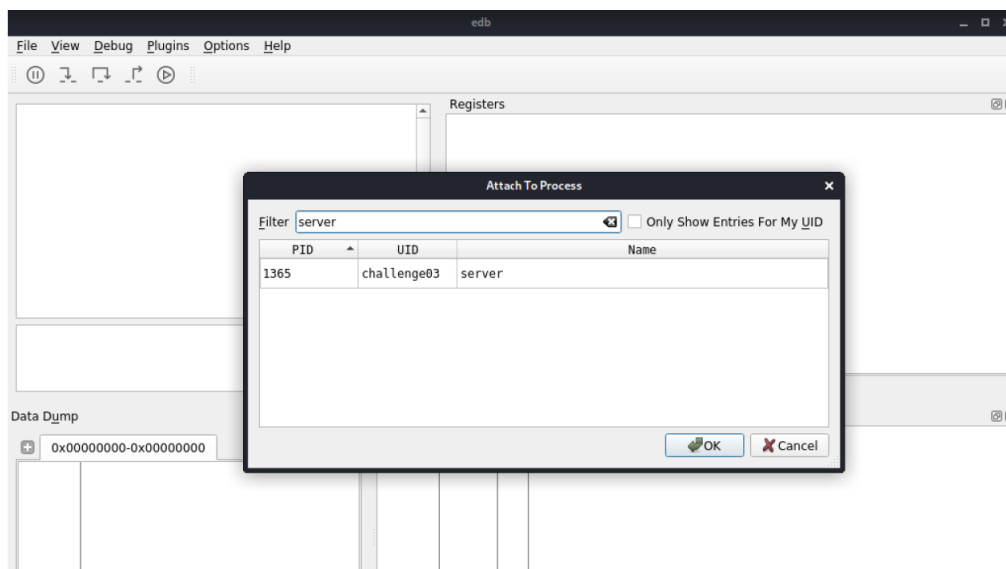


Figure 5: Click on File → Attach and select the server process

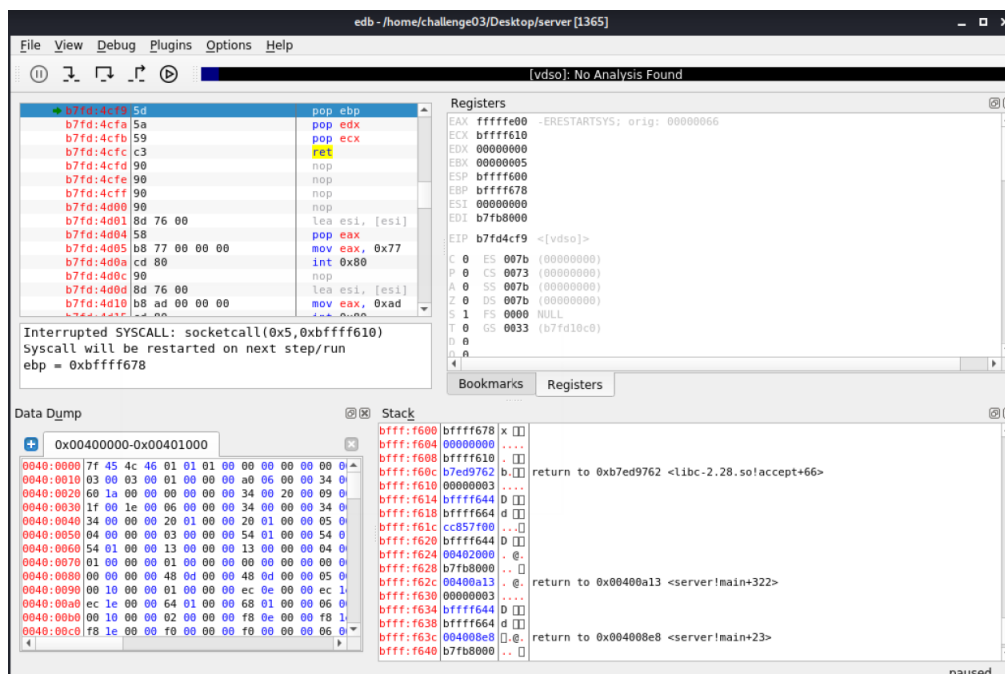


Figure 6: You can see the machine code on the top left, the Registers on the top right on the registers tab and the program stack on the bottom right.

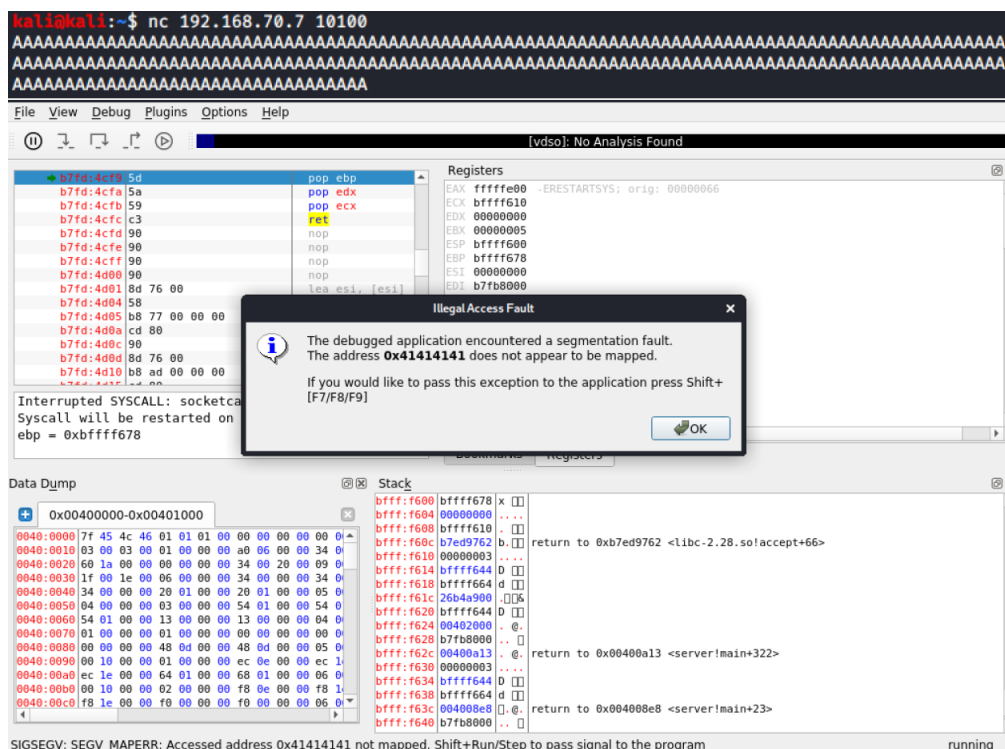


Figure 7: Now press play and test if the application is prone to a buffer overflow by sending a large string to the buffer.

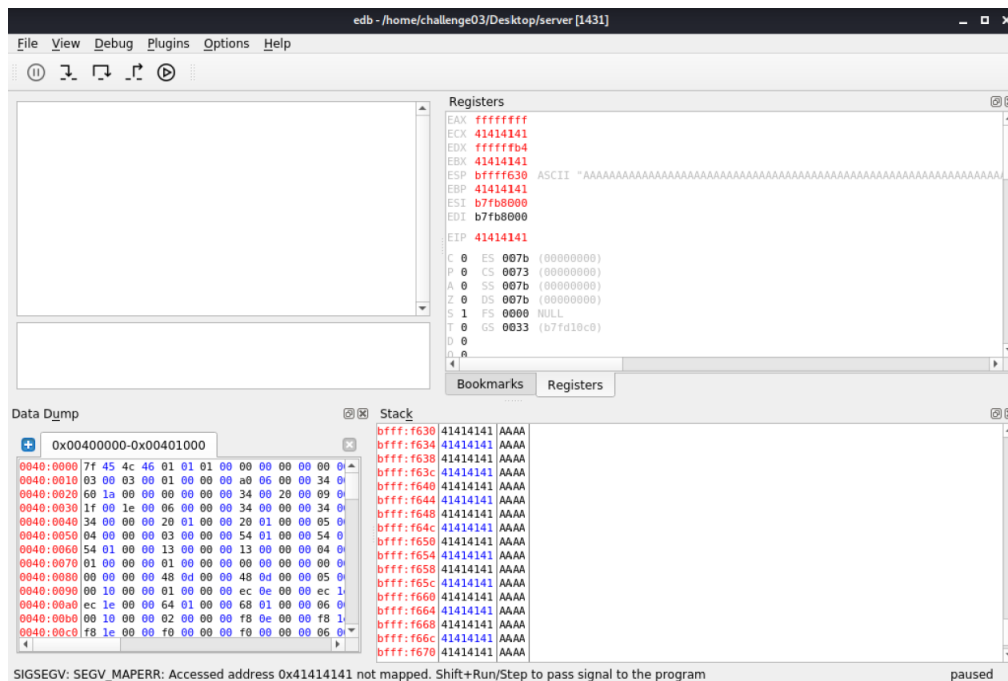


Figure 8: Now you can see that by writing over the buffer boundaries on the stack, important registers ECX, EBX, EBP and EIP were overwritten with the 'A's which are represented by the HEX ASCII value 41.